

Go Fiber + MySQL + GORM Implementation Plan

1. Objective

เอกสารนี้กำหนดแนวทางนำ Promotion Engine ที่ออกแบบไว้ไปพัฒนาเป็นระบบจริงด้วย:

Technology	Responsibility
Go	ภาษา Backend และ Core Business Logic
Fiber	HTTP API, Routing และ Middleware
MySQL	จัดเก็บ Product, Order, Promotion และ Audit Log
GORM	ORM, Repository, Association และ Transaction
Redis - Future	Cache Active Promotions

เป้าหมายของระบบ:

- คำนวณราคาสุทธิได้ถูกต้อง
- รองรับ Percentage และ Fixed Amount Promotion
- รองรับ Promotion Stacking และ Priority
- เพิ่ม Promotion ใหม่โดยไม่แก้ Calculation Flow หลัก
- มี Calculation Breakdown และ Audit Log
- พร้อมต่อยอดสู่ Production

2. Implementation Scope

2.1 MVP Scope

MVP ต้องประกอบด้วย:

1. Go Fiber HTTP Server
2. MySQL Database Connection ผ่าน GORM
3. Database Migration

4. Seed Product และ Promotion ตัวอย่าง
5. `POST /orders/calculate`
6. Product Repository
7. Promotion Repository
8. Pricing Service
9. Promotion Engine
10. Condition และ Target Evaluation
11. Deterministic Promotion Sorting
12. Stacking Policy
13. Strategy Factory
14. Percentage Discount Strategy
15. Fixed Amount Discount Strategy
16. Calculation Breakdown
17. Calculation Audit Log
18. Unit และ Integration Tests

2.2 Future Scope

ออกแบบรองรับไว้ แต่ยังไม่จำเป็นต้องพัฒนาใน MVP:

- Buy X Get Y
- Cart-level Discount
- Free Shipping
- Coupon Usage Locking
- Redis Cache
- Promotion Admin UI
- Authentication และ Authorization
- Async Audit Log

- Distributed Tracing

3. Project Structure

```
promotion-engine/
├── cmd/
│   └── server/
│       └── main.go
├── internal/
│   ├── config/
│   │   └── config.go
│   ├── database/
│   │   ├── mysql.go
│   │   └── migration.go
│   ├── handler/
│   │   ├── health_handler.go
│   │   ├── order_handler.go
│   │   └── promotion_handler.go
│   ├── service/
│   │   ├── pricing_service.go
│   │   ├── order_service.go
│   │   └── promotion_admin_service.go
│   ├── promotion/
│   │   ├── engine.go
│   │   ├── context.go
│   │   ├── result.go
│   │   ├── eligibility.go
│   │   ├── sorter.go
│   │   ├── stacking_policy.go
│   │   ├── constants.go
│   │   └── strategy/
│   │       ├── strategy.go
│   │       ├── registry.go
│   │       ├── percentage_discount.go
│   │       └── fixed_amount_discount.go
│   ├── repository/
│   │   ├── product_repository.go
│   │   ├── promotion_repository.go
│   │   ├── order_repository.go
│   │   └── calculation_log_repository.go
│   └── model/
│       ├── product.go
│       ├── order.go
│       ├── order_item.go
│       ├── promotion.go
│       ├── promotion_condition.go
│       ├── promotion_target.go
│       └── promotion_action.go
```

```

├── promotion_calculation_log.go
├── dto/
│   ├── calculate_order_request.go
│   ├── calculate_order_response.go
│   └── error_response.go
├── middleware/
│   ├── request_id.go
│   ├── logger.go
│   ├── recover.go
│   └── error_handler.go
├── seed/
│   └── seed.go
├── migrations/
├── tests/
├── .env.example
├── Dockerfile
├── docker-compose.yml
├── go.mod
└── README.md

```

4. Layer Responsibilities

Layer	Responsibility	Must Not Do
Handler	รับ Request และส่ง Response	คำนวณ Promotion
Service	ควบคุม Use Case	เขียน SQL โดยตรง
Promotion Engine	ควบคุม Calculation Flow	ผูกกับ Fiber หรือ GORM
Strategy	คำนวณ Discount แต่ละประเภท	Query Database
Repository	ติดต่อ Database ผ่าน GORM	มี Calculation Logic
Model	Map Database Table	ใช้เป็น API Response โดยตรง
DTO	กำหนด Request และ Response	มี Database Logic

Dependency ต้องไหลทางเดียว:

```

Handler
├── Service
├── Promotion Engine
├── Repository Interface
├── GORM Repository
└── MySQL

```

5. Application Bootstrap

`cmd/server/main.go` รับผิดชอบ:

1. โหลด Environment Configuration
2. เชื่อมต่อ MySQL
3. ตั้งค่า Connection Pool
4. รัน Migration
5. Register Strategies
6. Initialize Repositories
7. Initialize Services
8. Initialize Handlers
9. Register Routes และ Middleware
10. Start Fiber Server

ไม่ควรมี Business Logic ใน `main.go`

6. Configuration Plan

Environment Variables ที่ควรมี:

```
APP_ENV=development
APP_PORT=3000

DB_HOST=mysql
DB_PORT=3306
DB_USER=promotion_user
DB_PASSWORD=secret
DB_NAME=promotion_engine

DB_MAX_OPEN_CONNS=25
DB_MAX_IDLE_CONNS=10
DB_CONN_MAX_LIFETIME_MINUTES=30

CALCULATION_TIMEOUT_SECONDS=3
```

ข้อกำหนด:

- ห้าม Hardcode Credential
 - Production Secret ควรมาจาก Secret Manager
 - Validate Configuration ก่อน Start Server
-

7. Database Connection Plan

GORM ใช้สำหรับ:

- Model Mapping
- Association และ Preload
- Database Transaction
- Repository Implementation

Connection Pool ต้องกำหนด:

```
MaxOpenConns
MaxIdleConns
ConnMaxLifetime
```

เหตุผล:

- ป้องกัน MySQL Connection เต็ม
- รองรับ App หลาย Instance
- ควบคุม Resource Usage

ถ้าเชื่อมต่อ Database ไม่สำเร็จตอน Startup ระบบไม่ควรรับ Traffic

8. Migration Strategy

Production ควรใช้ Versioned Migration แทนการพึ่ง `AutoMigrate` เพียงอย่างเดียว

Migration ลำดับแรก:

```
001_create_product_tables
002_create_order_tables
003_create_promotion_tables
```

```
004_create_promotion_runtime_tables
005_create_indexes
```

แนวทาง:

- Development ใช้ AutoMigrate ได้
- Production ใช้ Versioned Migration
- Migration ต้องรันซ้ำได้อย่างปลอดภัย
- Seed Data ต้องแยกจาก Migration

9. GORM Model Plan

Model หลัก:

```
ProductCategory
Product
Order
OrderItem
Promotion
PromotionCondition
PromotionTarget
PromotionAction
PromotionUsage
PromotionCalculationLog
```

Association หลัก:

```
Promotion has many Conditions
Promotion has many Targets
Promotion has many Actions

Order has many OrderItems
Promotion has many PromotionUsages
```

หลักการ:

- เงินใช้ `int64` หน่วย satang
- Nullable Fields ใช้ Pointer หรือ Nullable Type

- Percentage ใช้ Basis Points
 - JSON Config ใช้ GORM JSON Datatype
 - Soft Delete ใช้เฉพาะ Config หรือ Resource ที่เหมาะสม
 - Transaction Records ไม่ควรถูกลบง่าย
-

10. Repository Implementation Plan

Product Repository

หน้าที่:

- โหลด Product หลายรายการใน Query เดียว
- ตรวจสอบ Product Status
- หลีกเลียง Query ราย Item

Method Concept:

```
FindActiveByIds(productIDs)
```

Promotion Repository

หน้าที่:

- โหลด Active Promotions
- Preload Conditions, Targets และ Actions
- เรียงข้อมูลเบื้องต้นแบบ Deterministic

Query Criteria:

```
status = ACTIVE  
starts_at <= now  
ends_at >= now
```

Calculation Log Repository

หน้าที่:

- บันทึก Calculation Snapshot
- ค้นหาด้วย Request ID หรือ Order ID

Order Repository

หน้าที่:

- สร้าง Order และ Order Items
- ทำงานภายใต้ Transaction ตอน Confirm Order

ข้อกำหนดสำคัญ:

Repository ต้องไม่คำนวณ Discount
Strategy ต้องไม่ Query Database

11. Promotion Engine Implementation Plan

Promotion Engine เป็น Core Business Logic และต้องไม่ผูกกับ Fiber หรือ GORM

Component ภายใน:

```
PromotionEngine
├─ CalculationContext
├─ EligibilityEvaluator
├─ PromotionSorter
├─ StackingPolicy
├─ StrategyRegistry
└─ CalculationResult
```

Engine Flow:

```
Receive Calculation Context
→ Load Active Promotions
→ Evaluate Eligibility
→ Sort Promotions
→ Check Stacking Policy
→ Select Strategy
→ Calculate and Apply Discount
```

→ Build Breakdown
→ Validate Result

12. Calculation Context Plan

`CalculationContext` ใช้เก็บ State ระหว่างคำนวณ:

```
User ID
Coupon Code
Items
Original Total
Discount Total
Final Total
Applied Promotions
Skipped Promotions
Stopped Targets / Scopes
```

แต่ละ Item ต้องเก็บ:

```
Product ID
Category ID
Quantity
Unit Price
Original Amount
Discount Amount
Final Amount
Applied Promotions
```

หลักสำคัญ:

- ห้ามแก้ Original Amount
- Promotion คำนวณจาก Remaining Amount
- ต้องเก็บ Before, Discount และ After ทุกครั้ง

13. Strategy Implementation Plan

MVP ต้องมี Strategy:

```
PercentageDiscountStrategy
FixedAmountDiscountStrategy
```

Strategy Registry Mapping:

```
PERCENTAGE_DISCOUNT → PercentageDiscountStrategy
FIXED_AMOUNT_DISCOUNT → FixedAmountDiscountStrategy
```

เมื่อเพิ่ม Promotion ใหม่:

1. เพิ่ม Strategy ใหม่
2. เพิ่ม Action Type
3. Register Strategy
4. เพิ่ม Config Validation
5. เพิ่ม Unit และ Regression Test

ไม่ต้องแก้ Promotion Engine Flow

14. Eligibility Implementation Plan

Eligibility Evaluator ต้องตรวจ:

```
Promotion Status
Date Range
Target Matching
Conditions
Coupon Requirement
Usage Limit Preview
```

MVP Condition ที่ควรรองรับ:

```
PRODUCT_ID
CATEGORY_ID
MIN_ORDER_AMOUNT
DATE_RANGE
```

Promotion ที่ไม่ผ่านต้องถูกเพิ่มใน `skipped_promotions` พร้อม Reason เช่น:

```
inactive
expired
not_started
target_not_matched
condition_not_matched
invalid_config
```

15. Sorting and Stacking Plan

Sort Order:

```
scope_order ASC
priority ASC
created_at ASC
promotion_id ASC
```

Scope Order:

```
ITEM → CART → COUPON → SHIPPING
```

Stacking Policy:

- `stackable=true` : ใช้ร่วมกับ Promotion อื่นได้
- `stackable=false` : Skip เมื่อมี Promotion ที่ขัดแย้งถูกใช้แล้ว
- `exclusive=true` : Stop Target หรือ Conflict Scope ที่เกี่ยวข้อง
- `stop_processing=true` : Stop Rule ถัดไปตาม Policy

สำหรับ ITEM Promotion ควรเก็บ Stop State ระดับ Item ไม่ควรหยุด Item Promotion ของสินค้าทั้ง Order

16. API Implementation Plan

Health Check

```
GET /healthz
GET /readyz
```

Calculate Order

```
POST /orders/calculate
```

หน้าที่:

- Validate Request
- โหลด Product Price จาก Server
- คำนวณ Promotion
- บันทึก Calculation Log
- Return Price และ Breakdown
- ไม่ Consume Promotion Usage

Confirm Order

```
POST /orders/confirm
```

หน้าที่:

- Recalculate Price
- Begin Transaction
- Lock Limited Promotions
- Validate Usage
- Create Order และ Order Items
- Consume Promotion Usage
- Save Calculation Snapshot
- Commit Transaction

Promotion Admin

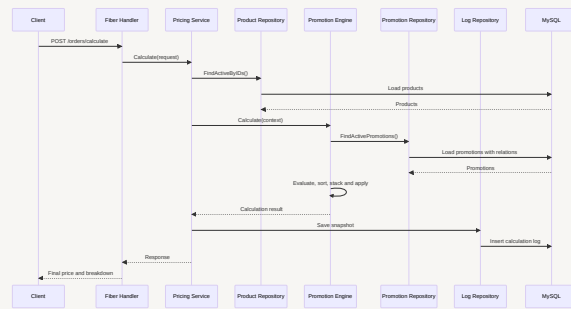
```
POST /promotions
GET /promotions
GET /promotions/:id
PATCH /promotions/:id
```

17. Calculate Order Runtime Flow

sequenceDiagram

```
participant C as Client
participant H as Fiber Handler
participant S as Pricing Service
participant P as Product Repository
participant E as Promotion Engine
participant R as Promotion Repository
participant L as Log Repository
participant DB as MySQL

C->>H: POST /orders/calculate
H->>S: Calculate(request)
S->>P: FindActiveByIDs()
P->>DB: Load products
DB-->>P: Products
S->>E: Calculate(context)
E->>R: FindActivePromotions()
R->>DB: Load promotions with relations
DB-->>R: Promotions
E->>E: Evaluate, sort, stack and apply
E-->>S: Calculation result
S->>L: Save snapshot
L->>DB: Insert calculation log
S-->>H: Response
H-->>C: Final price and breakdown
```

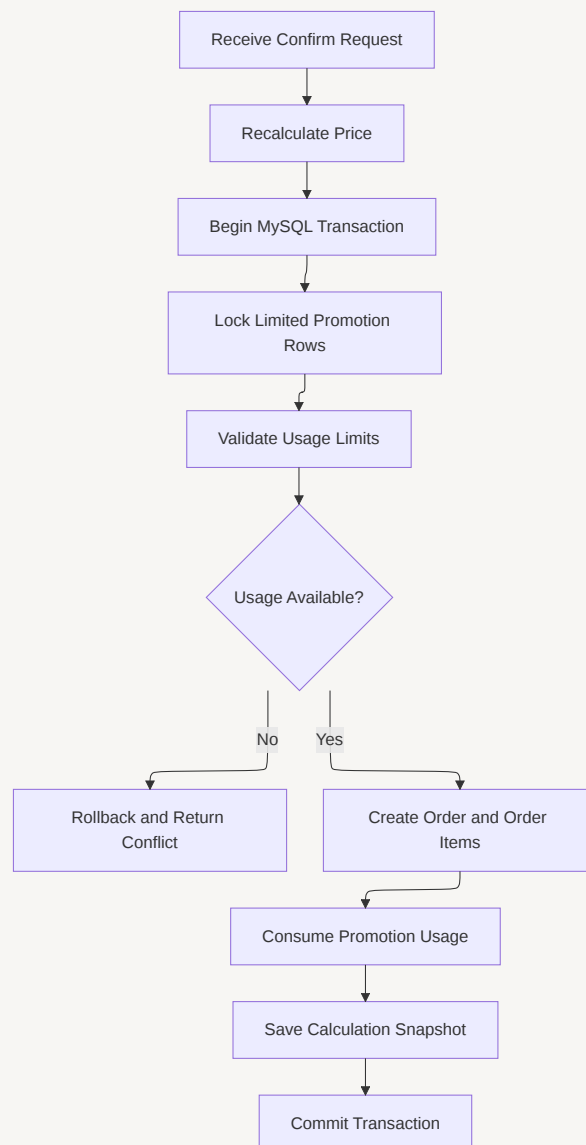


18. Confirm Order Transaction Flow

flowchart TD

```

A[Receive Confirm Request] --> B[Recalculate Price]
B --> C[Begin MySQL Transaction]
C --> D[Lock Limited Promotion Rows]
D --> E[Validate Usage Limits]
E --> F{Usage Available?}
F -- No --> G[Rollback and Return Conflict]
F -- Yes --> H[Create Order and Order Items]
H --> I[Consume Promotion Usage]
I --> J[Save Calculation Snapshot]
J --> K[Commit Transaction]
  
```



ต้องใช้ Primary Database และ Row-level Lock สำหรับ Promotion ที่มี Usage Limit

19. Error Handling Plan

Handler Errors

- Invalid JSON
- Missing Required Field
- Invalid Quantity

Service Errors

- Product Not Found
- Product Inactive
- Currency Mismatch
- Database Unavailable

Promotion Engine Skipped Reasons

- Strategy Not Found
- Invalid Promotion Config
- Condition Not Matched
- Target Not Matched
- Stacking Conflict
- Scope Stopped

บางกรณีไม่ควรทำให้ Request ล้ม แต่ควร Skip Promotion และบันทึก Reason Response ควรีรูปแบบเดียวกัน:

```
success
error.code
error.message
error.details
request_id
```

20. Logging and Observability Plan

ทุก Request ต้องมี `request_id`

Structured Log Events:

```
calculation_started
calculation_completed
promotion_applied
promotion_skipped
invalid_promotion_config
```

```
strategy_not_found
order_confirmed
promotion_usage_consumed
calculation_failed
```

Metrics สำคัญ:

```
promotion_calculation_latency
promotion_calculation_total
promotion_calculation_failed_total
promotion_applied_total
promotion_skipped_total
promotion_strategy_not_found_total
```

21. Testing Plan

Unit Tests

- Percentage Discount Strategy
- Fixed Amount Discount Strategy
- Discount Cap
- Eligibility Evaluator
- Promotion Sorter
- Stacking Policy

Integration Tests

- Repository + MySQL
- Pricing Service + Promotion Engine
- Calculation Log Persistence
- Confirm Order Transaction

API Tests

- Calculate Success

- Invalid Quantity
- Product Not Found
- Invalid Promotion Configuration

Concurrency Tests

- Coupon เหลือหนึ่งสิทธิ์ แต่มีหลาย Confirm Request
- ต้องมีเพียง Request เดียวที่สำเร็จ

Regression Tests

เมื่อเพิ่ม Strategy ใหม่ ต้องตรวจว่า Percentage, Fixed Amount, Stacking และ Exclusive เดิมยังทำงานถูกต้อง

22. Implementation Milestones

Milestone	Deliverables
1. Project Setup	Go Fiber Server, Config, Health Check
2. Database Setup	MySQL, GORM, Connection Pool, Migration
3. Core Data	Models, Repositories และ Seed Data
4. Engine Core	Context, Sorter, Eligibility และ Stacking
5. Basic Strategies	Percentage และ Fixed Amount
6. Calculate API	Final Price และ Breakdown
7. Auditability	Calculation Log และ Request ID
8. Confirm Order	Transaction และ Usage Locking
9. Verification	Unit, Integration, API และ Concurrency Tests
10. Production Review	Logging, Metrics, Cache และ Load Test

23. Recommended Implementation Order

1. Setup Go Module และ Fiber Server
2. เชื่อมต่อ MySQL ผ่าน GORM
3. สร้าง Versioned Migrations
4. สร้าง Models และ Repositories

5. Seed Product และ Promotion ตัวอย่าง
6. สร้าง DTO และ Calculate Handler
7. สร้าง Pricing Service
8. สร้าง Calculation Context
9. สร้าง Promotion Engine Core
10. สร้าง Eligibility, Sorter และ Stacking Policy
11. สร้าง Strategy Registry
12. สร้าง Percentage และ Fixed Strategies
13. สร้าง Calculation Breakdown และ Audit Log
14. เขียน Tests
15. เพิ่ม Confirm Order Transaction
16. ตรวจสอบ Production Readiness

24. Definition of Done

ระบบ MVP ถือว่าเสร็จเมื่อ:

- Server Start และเชื่อมต่อ MySQL ได้
- Migration ทำงานได้
- Product และ Promotion ถูกโหลดแบบ Batch
- ไม่มี Database Query ใน Calculation Loop
- Product 1 ลด 10% ได้ถูกต้อง
- Product 2 ลด 100 บาทได้ถูกต้อง
- รองรับ Priority และ Stacking
- Final Price ไม่ติดลบ
- Response มี Applied/Skipped Breakdown
- Calculation Log ถูกบันทึก
- Strategy Not Found ไม่ทำให้ระบบ Panic
- Unit และ Integration Tests ผ่าน
- เพิ่ม Strategy ใหม่ได้โดยไม่แก้ Engine Flow

Final Implementation Summary

ระบบเริ่มจาก **Modular Monolith** ด้วย Go Fiber, MySQL และ GORM โดยแบ่ง Handler, Service, Promotion Engine, Strategy และ Repository ออกจากกันอย่างชัดเจน

```
Client
→ Fiber Handler
→ Pricing Service
→ Product Repository
→ Promotion Engine
→ Eligibility / Sorter / Stacking Policy
→ Strategy Registry
→ Promotion Strategy
→ Calculation Log Repository
→ MySQL
```

แนวทางนี้ทำให้ระบบคำนวณ Promotion ได้ถูกต้อง ทดสอบง่าย เพิ่ม Promotion ใหม่ได้ โดยไม่กระทบ Logic เดิม และพร้อมต่อยอดด้วย Redis Cache, Transaction Locking และ Observability เมื่อเข้าสู่ Production.